LOGIN CON JWT

Comparación de hashes y emisión de tokens

Autenticación stateless en una API REST

Node.js · Express · MySQL · Bcrypt · JSON Web Token

Contenido

1. **El problema: ¿cómo sabe el servidor quién sos?** — *sesiones vs. tokens*
2. **Qué es un JWT: las tres partes** — *header, payload, signature*
3. **El recorrido completo del login** — *el handshake paso a paso*
4. **Paso 1: identificar al usuario** — *el modelo y la consulta parametrizada*
5. **Paso 2: la verificación criptográfica** — *bcrypt.compare()*
6. **Paso 3: el payload** — *qué datos viajan dentro del token*
7. **Paso 4: firmar el token** — *jwt.sign() y JWT_SECRET*
8. **Paso 5: entregar la llave al cliente** — *la respuesta HTTP*
9. **Cómo se usa el token** — *el header Authorization: Bearer*
10. **Paso 6: el middleware verificador** — *jwt.verify() como portero*
11. **Por qué esto es seguro** — *qué puede y qué no puede hacer un atacante*
12. **Los seis errores más comunes** — *y cómo evitarlos*
13. **Referencia** — *todo el código junto*
14. **Ejercicios propuestos** — *básico, intermedio y avanzado*

Cada concepto viene acompañado del código real del proyecto. Abrí los archivos mientras leés y ejecutá 'npm run demo' para ver el ciclo de vida del token en la consola.

1. El problema: ¿cómo sabe el servidor quién sos?

HTTP es un protocolo sin memoria. Cada petición que llega al servidor es independiente de todas las anteriores: cuando el navegador pide GET /api/usuarios/perfil, el servidor no tiene forma de saber quién está del otro lado. Es como un local con quinientos mozos y ningún mozo que recuerde a nadie.

La solución tradicional: sesiones

Cuando el usuario se loguea, el servidor guarda en su memoria (o en una tabla) una correspondencia entre un identificador de sesión y el usuario:

```
En la memoria del servidor
```

```
sesion_id: a8f3c1e9...   ->   usuario_id: 7
```

y le entrega al navegador una cookie con ese identificador. En cada petición posterior, el servidor busca el identificador en su memoria y así deduce quién es.

El problema aparece cuando hay que escalar. Si la aplicación corre en cinco servidores detrás de un balanceador de carga, el usuario puede loguearse contra el servidor 1 —que guarda la sesión— y su siguiente petición caer en el servidor 3, que no sabe absolutamente nada de esa sesión. Para resolverlo hay que compartir la memoria entre todos los servidores: Redis, una tabla en la base de datos, o configurar sesiones pegajosas. Es complejo y agrega puntos de falla.

La solución JWT: invertir el planteo

JSON Web Token propone lo contrario: en lugar de que el servidor recuerde quién sos, le entrega al usuario un comprobante firmado. El usuario lo muestra en cada petición, y el servidor solo tiene que comprobar que la firma sea válida.

> **ℹ Analogía**
> Pensá en un recital. Con sesiones, el portero tiene una lista con todos los que entraron y va tildando nombres. Con JWT, el portero te entrega una pulsera irreplicable: no te conoce, no te recuerda, pero reconoce su propia pulsera al instante. Podés entrar por cualquier puerta y cualquier portero te va a dejar pasar.

El servidor no guarda nada. Eso se llama autenticación stateless (sin estado) y es la razón por la que JWT se volvió el estándar en APIs REST y arquitecturas distribuidas.

> **✓ Idea clave**
> Con JWT el servidor no guarda la sesión: guarda la clave con la que firma. La información necesaria viaja dentro del propio token.

2. Qué es un JWT: las tres partes

Un JWT es una cadena de texto dividida en tres partes separadas por puntos.

Un token real emitido por el proyecto

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Nywicm9sIjoidXN1YXJpb19lc3RhbmRhciIsImlhdCI6MTc5
MTkyMzY0NSwiZXhwIjoxNzkxOTMwODQ1fQ.mJm9k9ooKEy1ubj__x7FK_zeWtxSsPEuSj72NkqjshQ
```

HEADER . **PAYLOAD** . **SIGNATURE**

Parte 1 — Header: el algoritmo

Indica con qué algoritmo se firmó el token.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

HS256 significa HMAC con SHA-256: un algoritmo de firma simétrica, es decir que la misma clave (JWT_SECRET) se usa tanto para firmar como para verificar.

Parte 2 — Payload: los datos

Contiene los datos que el servidor necesita recordar del usuario.

```
{
  "id": 7,
  "rol": "usuario_estandar",
  "iat": 1791923645,
  "exp": 1791930845
}
```

Los campos iat (issued at, emitido en) y exp (expiration, expira en) los agrega la librería automáticamente cuando se configura expiresIn. Son marcas de tiempo en formato Unix.

> ✕ **EL ERROR CONCEPTUAL**
> El payload NO está cifrado. Está codificado en base64.
> Base64 es una forma de representar datos, no una medida de seguridad. Cualquiera puede copiar un token, pegarlo en jwt.io y leer su contenido completo en dos segundos, sin ninguna clave.
> Por eso en el payload JAMÁS se incluye: contraseñas (ni siquiera hasheadas), documentos, datos personales sensibles, información que no quieras que sea pública.
> El JWT firma; no cifra. Garantiza que nadie pueda modificar el contenido, no que nadie pueda leerlo.

Parte 3 — Signature: el sello de seguridad

Esta es la parte que hace que todo el mecanismo sirva. Se calcula mezclando el header y el payload con la clave secreta que solo conoce el servidor.

```
HMACSHA256(
    base64url(header) + "." + base64url(payload),
    JWT_SECRET
)
```

¿Qué se logra con esto? Si un atacante cambia "rol": "usuario_estandar" por "rol": "admin", el payload cambia y, por lo tanto, el resultado de la fórmula también. El atacante no puede recalcular la nueva firma porque no conoce JWT_SECRET. Cuando el servidor verifica, recalcula la firma, ve que no coincide y rechaza el token.

Podés comprobarlo vos mismo ejecutando `npm run demo`, que intenta exactamente esos dos ataques y muestra el rechazo.

Salida real del script de demostración

```
PASO 5 - ATAQUE 1: alterar el payload
Un atacante cambia su rol a "admin" y reusa la firma vieja:
RECHAZADO -> JsonWebTokenError: invalid signature

PASO 6 - ATAQUE 2: firmar con una clave inventada
RECHAZADO -> JsonWebTokenError: invalid signature
```

> Lectura y modificación son dos cosas distintas. El payload es público (cualquiera lo lee) pero la firma lo hace inviolable (nadie lo modifica sin la clave).

3. El recorrido completo del login

Este es el mapa mental de todo el proceso. Conviene memorizarlo antes de mirar el código.

El “handshake” completo



Hay tres ideas que resumen este diagrama:

1. El login es la única vez que se envía la contraseña.
2. A partir de ahí, el cliente envía el token, no la contraseña.
3. El servidor nunca guardó que el usuario 7 estaba logueado: lo deduce del token en cada petición.

> El token reemplaza a la contraseña en todas las peticiones posteriores al login. Por eso el token es tan valioso: quien lo tenga, es el usuario.

4. Paso 1: identificar al usuario

El primer paso del login es buscar el usuario por su email. Esto ocurre en el controlador (`src/controllers/usuarioController.js`), que delega la consulta en el modelo.

Controlador: búsqueda por email

```
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    // ---- 1. Buscar al usuario en la BD ----
    const usuario = await usuarioModel.obtenerPorEmail(email);

    if (!usuario) {
      return res.status(401).json({ error: 'Credenciales invalidas.' });
    }
    // ...
```

Y en el modelo, que es la única capa que ejecuta SQL (src/models/usuarioModel.js):

```
const obtenerPorEmail = async (email) => {
  const query =
    'SELECT id, nombre, email, password, rol, estado_id ' +
    'FROM usuarios WHERE email = ? LIMIT 1';

  const [filas] = await pool.query(query, [email]);

  return filas.length > 0 ? filas[0] : null;
};
```

El signo de interrogación

Se usa consulta parametrizada en lugar de concatenar texto. La diferencia es la vulnerabilidad más explotada de las aplicaciones web: la inyección SQL.

```
// INCORRECTO: inyeccion SQL
const query = `SELECT * FROM usuarios WHERE email = '${email}'`;
// Si email = "' OR '1'='1", devuelve TODOS los usuarios

// CORRECTO: parametrizado
const query = 'SELECT * FROM usuarios WHERE email = ?';
await pool.query(query, [email]);
```

Con el `?` el driver de MySQL escapa el valor. Si alguien envía como email la cadena ' OR '1'='1, MySQL la busca como texto literal y no encuentra nada. Inyección neutralizada.

> En esta capa nunca se concatena una variable dentro del string de la consulta. Siempre placeholders '?' y los valores en un array aparte, en el mismo orden que los signos.

El mensaje de error genérico es intencional

```
// MAL: le dice al atacante que emails existen
if (!usuario) return res.status(401).json({ error: 'El email no existe.' });
if (!passValida) return res.status(401).json({ error: 'Contrasena incorrecta.' });

// BIEN: el mismo mensaje en los dos casos
if (!usuario) return res.status(401).json({ error: 'Credenciales invalidas.' });
if (!passValida) return res.status(401).json({ error: 'Credenciales invalidas.' });
```

¿Por qué? Porque si el mensaje cambia según el caso, un atacante puede probar diez mil emails y quedarse con la lista de los que sí existen. Esa lista ya es información valiosa: permite campañas de phishing dirigidas y ataques de fuerza bruta contra cuentas reales. La técnica se llama enumeración de usuarios y la defensa es la seguridad defensiva: no revelar nada.

Las pruebas automáticas del proyecto verifican exactamente esto. En `_tools/e2e.js` el caso 7 compara que el mensaje del email inexistente sea idéntico al de la contraseña incorrecta.

Una regla de negocio extra: el estado de la cuenta

```
// Autenticarse bien no alcanza: la cuenta debe estar Activa (id 1)
if (usuario.estado_id !== 1) {
  return res.status(403).json({
    error: 'La cuenta no esta activa. Contacte al administrador.'
  });
}
```

Acá se conecta directamente con la Fase 1 de la materia. La tabla estados_usuario tiene 1 = Activo, 2 = Inactivo y 3 = Suspendido. Un usuario suspendido puede escribir perfectamente su contraseña y aun así no debe poder entrar.

Fijate además el código HTTP elegido: **`403 Forbidden`** (sé quién sos, pero no tenés permiso) en lugar de **`401 Unauthorized`** (no sé quién sos). Los códigos HTTP no son adornos: comunican semántica y le permiten al frontend decidir qué hacer.

> El controlador no sabe SQL y el modelo no sabe de HTTP. El controlador orquesta; el modelo consulta. Esa separación es la clave del patrón MVC.

5. Paso 2: la verificación criptográfica

La pregunta inevitable del alumno es: si el hash es irreversible, **¿cómo se compara**?

Muchos buscan una función `bcrypt.decrypt()`. Esa función no existe, y si existiera, Bcrypt estaría roto: su razón de ser es precisamente que no se puede volver atrás.

La respuesta es que no se desencripta nada. Se vuelve a hashear la contraseña que acaba de escribir el usuario y se comparan los resultados.

Controlador: bcrypt compare

```
// ---- 2. Comparacion criptografica ----
const passValida = await bcrypt.compare(password, usuario.password);

if (!passValida) {
  return res.status(401).json({ error: 'Credenciales invalidas.' });
}
```

Qué ocurre por dentro

```
REGISTRO (lo que ya esta guardado en MySQL):

  "secreta123"  --bcrypt.hash(salt=10)-->  "$2b$10$rUzovtrHbR031Wa1n2yoS.o6J26W..."
                                                          |
                                                  (guardado en la BD)


LOGIN (lo que hace bcrypt.compare):

  "secreta123" --+
                 +--> extrae el salt del hash guardado
  "$2b$10$rUz..."+     y vuelve a hashear la contraseña plana
                       con ESE MISMO salt
                              |
                              v
                       "$2b$10$rUzovtrHbR031Wa1n2yoS.o6J26W..."
                              |
                       coincide con el guardado?
                              |
                        SI ---+--- NO
                         |          |
                   passValida   passValida
                     = true       = false
```

Las tres propiedades que hacen a Bcrypt el estándar

Propiedad	Qué significa	Por qué importa
Unidireccional	Del hash no se puede volver a la contraseña	Si roban la base de datos, no obtienen las claves
Salt automático	Se mezcla texto aleatorio antes de hashear	Dos usuarios con la misma clave tienen hashes distintos
Lento a propósito	El parámetro saltRounds ajusta el	La fuerza bruta se vuelve inviable en

	costo	tiempo y costo

El salt rompe las tablas precalculadas (*rainbow tables*). Sin salt, un atacante podría tener una tabla gigante con el hash de las contraseñas más comunes y buscarlas en un instante. Con salt, cada contraseña necesita su propio cálculo.

Anatomía del hash: por qué la columna es VARCHAR(255)

```
$2b$10$rUzovtrHbR031Wa1n2yoS.o6J26WwJkbFWOMeaIxYTlF35VSrKhqG
|__| |_| |____________________| |_____________________________|
  |    |           |                        |
  |    |          salt (22)              hash (31)
  |    +-- costo (10)
  +-- version de bcrypt ($2b$)

Total: 60 caracteres, siempre.
```

✕ El error silencioso

Si en la base de datos se declara password VARCHAR(50) en lugar de VARCHAR(255), MySQL corta el hash a 50 caracteres. El registro parece funcionar sin errores... pero el usuario nunca más va a poder iniciar sesión, porque bcrypt.compare() compara contra un hash truncado que ya no es válido.
Es especialmente peligroso porque no genera ningún mensaje de error: simplemente el login falla siempre. Por eso está en la Fase 1, en la validación estructural.

Dos trampas con async/await

bcrypt.compare() es asíncrona y devuelve una promesa. Por eso se usa `await`.

```
// GRAVÍSIMO: acepta CUALQUIER contraseña
const passValida = bcrypt.compare(password, usuario.password);
if (passValida) { /* una Promise siempre es truthy */ }

// CORRECTO
const passValida = await bcrypt.compare(password, usuario.password);
```

Si se omite el await, la variable passValida contiene un objeto Promise, y en JavaScript cualquier objeto es verdadero en una condición. El resultado es que el login acepta cualquier contraseña. Es un error de seguridad gravísimo y, además, muy fácil de cometer.

El mismo cuidado aplica a `bcrypt.hash()` en el registro: sin await se guardaría la promesa en lugar del hash.

bcrypt.compare() no desencripta: vuelve a hashear con el mismo salt y compara. Por eso el hash puede ser irreversible y la verificación funcionar igual.

6. Paso 3: el payload

Una vez verificada la contraseña, hay que decidir qué información va a viajar dentro del token.

```
// ---- 3. Firmar el JWT ----
const payload = {
  id: usuario.id,
  rol: usuario.rol
};
```

La regla del payload: lo mínimo necesario

El payload debe contener únicamente lo indispensable para identificar al usuario y autorizarlo. Nada más.

```
// CORRECTO: identificador y rol
{ id: 7, rol: 'usuario_estandar' }

// INCORRECTO: todo esto viaja VISIBLE en cada peticion
{
  id: 7,
  nombre: 'Ana',
  email: 'ana@mail.com',
  password: 'secreta123',        // NUNCA
  passwordHash: '$2b$10$...',    // tampoco: es publico
  dni: '30123456',               // dato personal
  tarjeta: '4111...'             // NUNCA
}
```

¿Por qué el id y no el objeto usuario completo?

Porque el payload viaja en cada petición. Imaginá un usuario haciendo cien peticiones por minuto: todo lo que pongas en el payload se transmite cien veces por minuto. Cuanto más chico, menos ancho de banda se consume y menos superficie de exposición hay. Si se necesitan más datos, se piden a la base de datos usando el id.

¿Por qué incluir el rol?

Para poder autorizar sin consultar la base de datos en cada petición.

```
const soloAdmin = (req, res, next) => {
  if (req.usuario.rol !== 'admin') {
    return res.status(403).json({ error: 'Acceso denegado.' });
  }
  next();
};
```

> ⚠ **El trade-off que hay que entender**
> Si el rol viaja en el token y a un usuario se le quita el rol de administrador, su token seguirá diciendo 'admin' hasta que expire. Es un intercambio consciente: se gana velocidad (no se consulta la base de datos en cada petición) y se pierde frescura. Para acciones críticas siempre conviene revalidar contra la base de datos.

Los campos que agrega la librería

```
{
  "id": 7,
  "rol": "usuario_estandar",
  "iat": 1791923645,      <- lo agrega jsonwebtoken
  "exp": 1791930845       <- lo agrega jsonwebtoken
}
```

No hay que calcularlos a mano: la librería los inserta automáticamente cuando se configura la opción expiresIn al firmar.

> Todo lo que se ponga en el payload es información pública. La pregunta correcta no es '¿me sirve tenerlo?' sino '¿me importaría que cualquiera lo lea?'.

7. Paso 4: firmar el token

La firma se centraliza en un único archivo, `src/utils/jwt.js`. La razón es que la clave secreta es una sola para todo el proyecto: si se repite en cinco archivos, tarde o temprano uno queda desactualizado y el login deja de funcionar.

src/utils/jwt.js

```
const jwt = require('jsonwebtoken');
require('dotenv').config();

const JWT_SECRET = process.env.JWT_SECRET;
const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || '2h';

// Falla rapido y claro si falta la clave secreta.
if (!JWT_SECRET) {
  throw new Error('Falta JWT_SECRET en el archivo .env.');
}

function firmarToken(payload) {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
}

function verificarToken(token) {
  return jwt.verify(token, JWT_SECRET);
}
```

Decisión 1: la clave vive en .env, nunca en el código

```
// NUNCA: cualquiera que vea el repositorio tiene la llave maestra
const JWT_SECRET = 'mi_clave_super_secreta';

// SIEMPRE
const JWT_SECRET = process.env.JWT_SECRET;
```

Con la `JWT_SECRET` un atacante puede fabricar tokens válidos para cualquier usuario, incluidos los administradores. Es el activo más valioso del sistema. Por eso el archivo .env está listado en .gitignore desde la Fase 2: nunca se sube a Git.

Para generar una clave decente:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Decisión 2: expiresIn es obligatorio

```
jwt.sign(payload, JWT_SECRET, { expiresIn: '2h' })
```

Sin expiración, un token robado es válido para siempre. Con 2h, el daño queda acotado a dos horas: después el usuario debe volver a loguearse. La librería agrega el campo exp automáticamente.

Los formatos aceptados son cadenas legibles: '15m' (quince minutos), '2h' (dos horas), '7d' (siete días).

Decisión 3: fallar rápido si falta la configuración

```
if (!JWT_SECRET) {
  throw new Error('Falta JWT_SECRET en el archivo .env.');
}
```

Sin esta verificación, el token se firmaría con una clave indefinida y el error aparecería mucho después, en forma de un mensaje críptico difícil de rastrear. Es mejor que el servidor no arranque a que arranque inseguro.

Qué devuelve jwt.sign()

Una cadena de texto: el token completo. Eso es todo. No es un objeto, no es un JSON, no hay que serializarlo. Es una cadena que se envía tal cual al cliente.

> JWT_SECRET es la llave maestra del sistema. Si se filtra, la autenticación deja de existir. Vive en .env, es larga, es aleatoria y se rota periódicamente.

8. Paso 5: entregar la llave al cliente

```
// ---- 4. Entregar la llave ----
return res.status(200).json({
  mensaje: 'Login exitoso.',
  token,
  expira_en: JWT_EXPIRES_IN,
  usuario: {
    id: usuario.id,
    nombre: usuario.nombre,
    email: usuario.email,
    rol: usuario.rol
  }
});
```

Respuesta real, capturada de la API de este proyecto:


```
{
  "mensaje": "Login exitoso.",
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6OCwicm9sIjoidXN1YXJpb19lc3RhbmRhciIsImlhdCI6MTc
5MTkyMzc4NywiZXhwIjoxNzkxOTMwOTg3fQ.mJm9k9ooKEy1ubj__x7FK_zeWtxSsPEuSj72NkqjshQ",
  "expira_en": "2h",
  "usuario": {
    "id": 8,
    "nombre": "Alumno de Prueba",
    "email": "alumno786673@mail.com",
    "rol": "usuario_estandar"
  }
}
```


Qué NO debe devolver nunca el login

Campo	¿Se devuelve?	Motivo
token	SÍ	Es el objetivo de la petición
id, nombre, email, rol	SÍ	Datos públicos útiles para el frontend
password (hash)	NO	Nunca. Es material sensible aunque sea un hash
password (texto plano)	NO	Jamás. No existe en ningún momento
estado_id / estado	Opcional	Puede ser útil, pero no es necesario

Si el endpoint devuelve el objeto usuario tal como salió de la base de datos, está filtrando el hash.

¿Dónde guarda el token el cliente?

Opción	Ventaja	Riesgo
localStorage	Simple; sobrevive al cierre del navegador	Vulnerable a XSS: si inyectan JavaScript, roban el token
Cookie httpOnly	Inmune a XSS: el JavaScript no puede leerla	Requiere configurar CORS y credentials
Memoria (variable)	Lo más seguro contra XSS	Se pierde al recargar la página

Para el alcance de esta materia se usa localStorage, que es lo más didáctico. Conviene saber que en producción con requisitos de seguridad altos se prefiere la cookie httpOnly acompañada de SameSite=Strict.

El login devuelve el token y la información pública del usuario. El hash de la contraseña nunca sale de la base de datos.

9. Cómo se usa el token

Una vez que el cliente tiene el token, lo envía en el header Authorization con el prefijo Bearer.

```
GET /api/usuarios/perfil HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

`Bearer` significa portador: quien porta este token tiene acceso. Es el estándar definido en el RFC 6750 y no es opcional: sin esa palabra el servidor responde 401.

Los tres formatos, uno solo correcto

```
Authorization: Bearer <token>      CORRECTO
Authorization: <token>             INCORRECTO -> 401
authorization: bearer <token>      HTTP/1.1 ignora mayusculas en el nombre
```

Guardar y usar el token en el frontend

```
// Despues del login, guardar el token
localStorage.setItem('token', data.token);

// En cada peticion protegida, adjuntarlo
const token = localStorage.getItem('token');

const respuesta = await fetch('http://localhost:3000/api/usuarios/perfil', {
  headers: {
    'Authorization': `Bearer ${token}`
  }
});

if (respuesta.status === 401) {
  // El token expiro o es invalido: volver al login
  localStorage.removeItem('token');
  window.location.href = '/login.html';
}
```

> ⚠ **Detalle de diseño**
> El frontend debe manejar el 401 de forma centralizada. Si cada petición tiene su propio manejo de errores, tarde o temprano alguna se olvida de redirigir al login y el usuario queda viendo una pantalla vacía sin entender por qué.

Cómo probarlo en Postman

4. Pestaña Authorization.
5. Type: Bearer Token.
6. Pegar el token en el campo Token.

Postman arma el header automáticamente. En el capítulo de pruebas de este proyecto hay una manera para que Postman guarde el token solo, usando la pestaña Tests con pm.environment.set("token", respuesta.token).

A partir del login, el token reemplaza a la contraseña. El header Authorization: Bearer <token> es el nuevo 'usuario y contraseña'.

10. Paso 6: el middleware verificador

Un **middleware** es una función que se ejecuta antes del controlador. Es un filtro. Recibe tres argumentos:

Argumento	Qué es	Para qué sirve
req	La petición	Leer headers, body, parámetros
res	La respuesta	Devolver un resultado al cliente
next	El paso siguiente	Llamarla da paso al controlador

Si el middleware **no llama a next()**, la petición muere ahí. Así se corta el acceso.

`src/middlewares/authMiddleware.js`

```
const { verificarToken } = require('../utils/jwt');

function verificarAuth(req, res, next) {
  const authHeader = req.headers['authorization'];

  // 1. Sin header no hay nada que verificar.
  if (!authHeader) {
    return res.status(401).json({ error: 'Token no proporcionado.' });
  }

  // 2. El estandar es: "Bearer eyJhbGciOi..."
  const partes = authHeader.split(' ');

  if (partes.length !== 2 || partes[0] !== 'Bearer') {
    return res.status(401).json({
      error: 'Formato de token invalido. Use: Bearer <token>'
    });
  }

  const token = partes[1];

  // 3. Verificacion criptografica de la firma + expiracion.
  try {
    const payload = verificarToken(token);
    req.usuario = payload;   // <-- aca se inyecta la identidad
    return next();           // <-- aca se da paso al controlador
  } catch (error) {
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({
        error: 'El token expiro. Inicie sesion nuevamente.'
      });
    }
    return res.status(403).json({ error: 'Token invalido o firma alterada.' });
  }
}
```

El momento clave: req.usuario = payload

Esta línea es la que hace que todo funcione. El middleware enriquece la petición con la identidad. A partir de ahí, cualquier controlador puede hacer:

```
const perfil = async (req, res) => {
  const usuario = await usuarioModel.obtenerPorId(req.usuario.id);
  //                                                ^^^^^^^^^^^^^^^
  //                    Viene del token, no de un parametro de la URL
  return res.status(200).json({ usuario });
};
```

Comparalo con la alternativa insegura:

```
// MAL: cualquiera puede pedir /perfil/1 y ver los datos de otro usuario
router.get('/perfil/:id', usuarioController.perfil);
```

Con el middleware, el `id` no se puede falsificar: viene del token firmado. Ese es el valor real de toda esta maquinaria.

Los códigos de error, bien elegidos

Situación	Código	Significado
No envió el header	401	No sé quién sos
Formato Bearer mal escrito	401	No sé quién sos
Token expirado	401	Tu credencial ya no sirve; volvé a loguearte
Firma alterada	403	La credencial es falsa: no es un tema de vencimiento
Cuenta inactiva	403	Sé quién sos, pero no tenés permiso

La distinción entre 401 y 403 permite al frontend decidir si debe redirigir al login o mostrar un error de seguridad.

Cómo se conecta con las rutas

`src/routes/usuarioRoutes.js`

```
// ---- Rutas publicas (no requieren token) ----
router.post('/registro', usuarioController.registrar);
router.post('/login', usuarioController.login);

// ---- Ruta protegida (requiere JWT valido) ----
router.get('/perfil', verificarAuth, usuarioController.perfil);
//                    ^^^^^^^^^^^^
//         El middleware se ejecuta ANTES del controlador
```

Esa coma es toda la seguridad de la ruta. Express ejecuta verificarAuth y solo si llama a next() llega a usuarioController.perfil. Agregar un endpoint protegido es tan simple como intercalar ese middleware.

> El middleware es el portero. Verifica la firma, inyecta la identidad en req.usuario y da paso. El controlador nunca vuelve a preguntar quién es el usuario: ya lo sabe.

11. Por qué esto es seguro

Lo que un atacante NO puede hacer

Ataque	Resultado	Por qué
Leer el payload	SÍ puede	Está en base64, no cifrado. Por eso no se ponen datos sensibles
Cambiarse el rol a admin	Imposible	La firma se invalida; necesitaría JWT_SECRET
Fabricar un token desde cero	Imposible	Sin JWT_SECRET la firma no valida
Usar el token después de expirar	Imposible	jwt.verify() controla el campo exp
Adivinar contraseñas por los mensajes	Imposible	Los mensajes de error son genéricos
Inyectar SQL en el login	Imposible	Se usan consultas parametrizadas con '?'

Lo que SÍ es un riesgo real

Riesgo	Qué pasa	Mitigación
Robo del token (XSS)	Si inyectan JavaScript en la página, leen localStorage	Cookies httpOnly; escapar todo el HTML
JWT_SECRET filtrado	El atacante fabrica cualquier token	Clave larga en .env, rotación periódica
Token en la URL	Los logs y el historial lo guardan	Siempre en el header Authorization
Sin HTTPS	El token viaja en texto plano y lo interceptan	HTTPS obligatorio en producción
Tokens sin expiración	Un token robado sirve para siempre	expiresIn siempre configurado
Logout	Con JWT puro no se puede invalidar antes de expirar	Refresh tokens o lista negra

Si el servidor no guarda sesiones, ¿cómo se implementa un logout real?
¿Y si a un usuario le roban el token y le cambian la contraseña: el token viejo sigue siendo válido?
¿Es aceptable? ¿Cómo lo resolverías? Pensá qué trade-off introduce cada solución: cada mecanismo para invalidar tokens obliga al servidor a volver a guardar estado, que es justamente lo que JWT vino a evitar.

El JWT resuelve la autenticación de forma elegante, pero no es a prueba de balas. Hay que entender sus límites para decidir cuándo conviene y cuándo no.

12. Los seis errores más comunes

1. Olvidar el await en bcrypt.compare

Síntoma: El login acepta CUALQUIER contraseña, porque una promesa siempre es verdadera en una condición.

MAL

```
const passValida = bcrypt.compare(password, usuario.password);
```

BIEN

```
const passValida = await bcrypt.compare(password, usuario.password);
```

2. Guardar el token pero no enviarlo

Síntoma: Toda ruta protegida responde 401, aunque el login haya funcionado.

MAL

```
fetch('/api/usuarios/perfil');
```

BIEN

```
fetch('/api/usuarios/perfil', {
  headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
});
```

3. Poner la contraseña en el payload

Síntoma: Cualquiera que copie el token lo lee en jwt.io.

MAL

```
jwt.sign({ id, email, password }, SECRET);
```

BIEN

```
jwt.sign({ id, rol }, SECRET);
```

4. Declarar password como VARCHAR(50)

Síntoma: El registro parece funcionar pero el login falla siempre, sin ningún error.

MAL

```
password VARCHAR(50)   -- el hash se trunca
```

BIEN

```
password VARCHAR(255)  -- correcto
```

5. JWT_SECRET distinto al firmar y al verificar

Síntoma: El login devuelve un token, pero TODA ruta protegida responde 403 'invalid signature'.

MAL

```
const SECRET = 'clave1';  // en un archivo
const SECRET = 'clave2';  // en otro archivo
```

BIEN

```
// Un unico archivo: src/utils/jwt.js
```

6. Confundir 401 con 403

Síntoma: El frontend no sabe si debe redirigir al login o mostrar un error de permisos.

MAL

```
401: sin token / expirado
403: firma alterada / sin permiso
```

BIEN

```
401 = no estás autenticado
403 = estás autenticado pero no autorizado
```

13. Referencia

Registro (una sola vez)

```
const hash = await bcrypt.hash(password, 10);   // 60 chars, irreversible

await pool.query(
  'INSERT INTO usuarios (nombre, email, password) VALUES (?, ?, ?)',
  [nombre, email, hash]        // rol y estado_id los pone la BD
);
```

Login (devuelve el token)

```
const usuario = await usuarioModel.obtenerPorEmail(email);
if (!usuario) {
  return res.status(401).json({ error: 'Credenciales invalidas.' });
}

const ok = await bcrypt.compare(password, usuario.password);   // await!
if (!ok) {
  return res.status(401).json({ error: 'Credenciales invalidas.' });
}

const token = jwt.sign(                    // firmar
  { id: usuario.id, rol: usuario.rol },    // payload publico
  process.env.JWT_SECRET,                  // secreto del .env
  { expiresIn: '2h' }                      // caducidad
);

return res.status(200).json({ token });
```

Middleware (protege las rutas)

```
const authHeader = req.headers['authorization'];
if (!authHeader) {
  return res.status(401).json({ error: 'Token no proporcionado.' });
}

const token = authHeader.split(' ')[1];
try {
  req.usuario = jwt.verify(token, process.env.JWT_SECRET);
  next();
} catch (e) {
  const codigo = e.name === 'TokenExpiredError' ? 401 : 403;
  return res.status(codigo).json({ error: 'Token invalido.' });
}
```

Cliente

```
localStorage.setItem('token', data.token);

fetch('/api/usuarios/perfil', {
  headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
});
```

Instalación y variables de entorno

```
npm install express mysql2 dotenv cors jsonwebtoken bcryptjs
npm install --save-dev nodemon
```

.env

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_password
DB_NAME=sistema_web_db

JWT_SECRET=clave_larga_y_aleatoria
JWT_EXPIRES_IN=2h
BCRYPT_SALT_ROUNDS=10
```

1. El JWT firma, no cifra. Cualquiera lo lee; nadie lo modifica.
2. El payload es público. Nunca datos sensibles adentro.
3. bcrypt.compare no desencripta: vuelve a hashear.
4. JWT_SECRET es la llave maestra. Vive en .env, nunca en el código.
5. El servidor no recuerda nada. Esa es la ventaja y también el precio.

14. Ejercicios propuestos

Nivel básico

1. Ejecutá npm run demo y explicá con tus palabras qué ocurre en el PASO 5.
2. Copiá un token real del login y pegalo en jwt.io. ¿Qué campos ves? ¿Por qué no aparece la contraseña?
3. Modificá JWT_EXPIRES_IN a '30s', reiniciá la API, logueate y esperá treinta segundos. Hacé GET /api/usuarios/perfil. ¿Qué código HTTP recibís? ¿Qué mensaje?
4. Registrá un usuario y consultá la tabla usuarios en MySQL Workbench. ¿Qué valor tiene la columna rol? ¿Y estado_id? ¿Quién los asignó?

Nivel intermedio

5. Creá el endpoint GET /api/usuarios/admin/lista que devuelva todos los usuarios. Debe ser accesible solo si req.usuario.rol === 'admin'. Pista: creá un middleware soloAdmin y encadenalo después de verificarAuth.
6. Implementá POST /api/usuarios/logout y explicá qué debería hacer. ¿Puede el servidor invalidar el token que ya entregó? ¿Qué alternativas existen?
7. Agregá el cambio de contraseña en PUT /api/usuarios/password como ruta protegida. Debe pedir la contraseña actual, compararla con bcrypt.compare, hashear la nueva y actualizarla. ¿Qué debería pasar con el token que el usuario ya tenía en la mano?
8. Agregá una columna rol_admin a la respuesta del login y verificá que el frontend pueda usarla para mostrar u ocultar el botón de administración. ¿Es suficiente eso como control de seguridad? ¿Por qué no?

15. Repaso de contenidos

1. ¿Por qué el hash de la contraseña no se puede desencriptar?

Respuesta: Porque Bcrypt es una función unidireccional: aplica operaciones matemáticas que no tienen inversa práctica. Comparar no requiere desencriptar, sino volver a hashear y comparar resultados.

2. ¿Qué pasa si guardo la contraseña en el payload del token?

Respuesta: Queda expuesta: el payload está en base64, no cifrado, así que cualquiera que vea el token puede leerla en jwt.io.

3. ¿Dónde se guarda el JWT_SECRET y por qué?

Respuesta: En el archivo .env, que está excluido del control de versiones. Es la clave con la que se firman los tokens: quien la conozca puede fabricar tokens válidos para cualquier usuario.

4. ¿Qué diferencia hay entre 401 y 403?

Respuesta: 401 significa 'no sé quién sos' (falta el token, expiró o es inválido). 403 significa 'sé quién sos, pero no podés hacer esto' (firma alterada, rol insuficiente, cuenta inactiva).

5. ¿Por qué el mensaje de error del login es siempre el mismo?

Respuesta: Para evitar la enumeración de usuarios. Si el mensaje distinguiera entre 'email inexistente' y 'contraseña incorrecta', un atacante podría descubrir qué emails están registrados.

6. ¿Qué hace exactamente el middleware?

Respuesta: Extrae el token del header Authorization, verifica su firma y su expiración con jwt.verify(), inyecta el payload en req.usuario y llama a next() para dar paso al controlador. Si algo falla, corta la petición con 401 o 403.

7. ¿Por qué la columna password debe ser VARCHAR(255)?

Respuesta: Porque un hash Bcrypt mide 60 caracteres. Con un tamaño menor el hash se trunca, el registro parece exitoso y el login falla siempre, sin ningún mensaje de error.

8. ¿Por qué se usa el placeholder '?' en las consultas SQL?

Respuesta: Para prevenir inyección SQL. El driver escapa el valor, así que una entrada maliciosa se trata como texto literal y no como código SQL.

9. ¿Qué significa que la autenticación JWT sea stateless?

Respuesta: Que el servidor no guarda ninguna sesión. Toda la información necesaria viaja dentro del token firmado, y el servidor solo verifica la firma en cada petición.

10. ¿Por qué el token debe tener fecha de expiración?

Respuesta: Para acotar el daño si el token es robado. Sin expiración, un token filtrado sirve para siempre; con expiresIn, el usuario debe volver a loguearse periódicamente.

El JWT no es magia: es una cadena de texto firmada criptográficamente. Una vez que se entiende que firma no es lo mismo que cifra, y que el servidor no guarda estado, todo el resto encaja.
La mejor forma de internalizarlo es romperlo: alterá un token, cambiale la clave al servidor, dejalo expirar. Ver los errores es lo que hace que se entienda.